Proyecto final - Fundamentos de Hardware

Adrián Barbudo López y Marcos Rodriguez Fernández

```
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
import os
import shutil
import psutil
import platform
from datetime import datetime

class FileManagerApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Gestor de Archivos y Panel de Control")
        self.root.geometry("900x600")

        self.style = ttk.Style()
        self.style.theme_use('clam')

        self.current_dir = os.getcwd()

        self.create_widgets()
        self.update_file_list()
        self.update_system_stats()

    def create_widgets(self):
        self.menu_bar = tk.Menu(self.root)
        self.root.config(menu=self.menu_bar)

        file_menu = tk.Menu(self.menu_bar, tearoff=0)
        self.menu_bar.add_cascade(label="Archivo", menu=file_menu)
        file_menu.add_command(label="Cambiar Directorio",
command=self.change_directory)
        file_menu.add_separator()
        file_menu.add_command(label="Salir", command=self.root.quit)

        self.main_paned = ttk.Panedwindow(self.root,
orient=tk.HORIZONTAL)
        self.main_paned.pack(fill=tk.BOTH, expand=True)
```

```
        self.file_frame = ttk.Frame(self.main_paned, padding="10")
        self.main_paned.add(self.file_frame, weight=3)

        ttk.Label(self.file_frame, text="Archivos en el Directorio:",
font=('Arial', 10, 'bold')).pack(anchor=tk.W)

        search_frame = ttk.Frame(self.file_frame)
        search_frame.pack(fill=tk.X, pady=5)
        ttk.Label(search_frame, text="Buscar:").pack(side=tk.LEFT)
        self.search_var = tk.StringVar()
        self.search_var.trace_add("write", lambda *args:
self.filter_files())
        ttk.Entry(search_frame,
textvariable=self.search_var).pack(side=tk.LEFT, fill=tk.X,
expand=True, padx=5)

        self.tree = ttk.Treeview(self.file_frame, columns=("Tipo",
"Tamaño", "Modificado"), show='headings')
        self.tree.heading("Tipo", text="Tipo")
        self.tree.heading("Tamaño", text="Tamaño")
        self.tree.heading("Modificado", text="Última Modificación")
        self.tree.column("Tipo", width=100)
        self.tree.pack(fill=tk.BOTH, expand=True)

        btn_frame = ttk.Frame(self.file_frame)
        btn_frame.pack(fill=tk.X, pady=10)

        ttk.Button(btn_frame, text="Nueva Carpeta",
command=self.create_folder).pack(side=tk.LEFT, padx=2)
        ttk.Button(btn_frame, text="Renombrar",
command=self.rename_item).pack(side=tk.LEFT, padx=2)
        ttk.Button(btn_frame, text="Mover",
command=self.move_item).pack(side=tk.LEFT, padx=2)
        ttk.Button(btn_frame, text="Eliminar",
command=self.delete_item).pack(side=tk.LEFT, padx=2)

        self.stats_frame = ttk.LabelFrame(self.main_paned, text=" Panel
de Control ", padding="10")
        self.main_paned.add(self.stats_frame, weight=1)

        self.cpu_label = ttk.Label(self.stats_frame, text="CPU: 0%")
        self.cpu_label.pack(pady=5, anchor=tk.W)
```

```
        self.ram_label = ttk.Label(self.stats_frame, text="RAM: 0%")
        self.ram_label.pack(pady=5, anchor=tk.W)

        self.disk_label = ttk.Label(self.stats_frame, text="Disco
Libre: ---")
        self.disk_label.pack(pady=5, anchor=tk.W)

        self.file_count_label = ttk.Label(self.stats_frame,
text="Archivos: 0")
        self.file_count_label.pack(pady=5, anchor=tk.W)

        self.cores_label = ttk.Label(self.stats_frame, text=f"Núcleos:
{psutil.cpu_count()}")
        self.cores_label.pack(pady=5, anchor=tk.W)

        self.battery_label = ttk.Label(self.stats_frame, text="Batería:
N/A")
        self.battery_label.pack(pady=5, anchor=tk.W)

    def update_file_list(self):
        for i in self.tree.get_children():
            self.tree.delete(i)

        try:
            items = os.listdir(self.current_dir)
            self.file_count_label.config(text=f"Archivos:
{len(items)}")

            for item in items:
                full_path = os.path.join(self.current_dir, item)
                tipo = "Carpeta" if os.path.isdir(full_path) else
"Archivo"
                size = f"{os.path.getsize(full_path) / 1024:.1f} KB"
                mtime =
datetime.fromtimestamp(os.path.getmtime(full_path)).strftime('%Y-%m-%d
%H:%M')

                self.tree.insert("", tk.END, iid=item, values=(tipo,
size, mtime), text=item)
        except Exception as e:
            messagebox.showerror("Error", f"No se pudo leer el
directorio: {e}")
```

```
    def filter_files(self):
        query = self.search_var.get().lower()
        for item in self.tree.get_children():
            if query in item.lower():
                self.tree.reattach(item, "", tk.END)
            else:
                self.tree.detach(item)

    def change_directory(self):
        new_dir = filedialog.askdirectory()
        if new_dir:
            self.current_dir = new_dir
            self.update_file_list()

    def create_folder(self):
        name = self.prompt_dialog("Nueva Carpeta", "Nombre de la
carpeta:")
        if name:
            try:
                os.mkdir(os.path.join(self.current_dir, name))
                self.update_file_list()
            except Exception as e:
                messagebox.showerror("Error", str(e))

    def delete_item(self):
        selected = self.tree.focus()
        if not selected: return

        if messagebox.askyesno("Confirmar", f"¿Eliminar
'{selected}'?"):
            try:
                path = os.path.join(self.current_dir, selected)
                if os.path.isdir(path):
                    shutil.rmtree(path)
                else:
                    os.remove(path)
                self.update_file_list()
            except Exception as e:
                messagebox.showerror("Error", str(e))

    def rename_item(self):
        selected = self.tree.focus()
        if not selected: return
```

```
        new_name = self.prompt_dialog("Renombrar", f"Nuevo nombre para
{selected}:")
        if new_name:
            try:
                os.rename(os.path.join(self.current_dir, selected),
                          os.path.join(self.current_dir, new_name))
                self.update_file_list()
            except Exception as e:
                messagebox.showerror("Error", str(e))

    def move_item(self):
        selected = self.tree.focus()
        if not selected: return

        dest = filedialog.askdirectory(title="Seleccionar destino")
        if dest:
            try:
                shutil.move(os.path.join(self.current_dir, selected),
dest)
                self.update_file_list()
            except Exception as e:
                messagebox.showerror("Error", str(e))

    def update_system_stats(self):
        cpu_usage = psutil.cpu_percent()
        ram_usage = psutil.virtual_memory().percent

        self.cpu_label.config(text=f"CPU: {cpu_usage}%")
        self.ram_label.config(text=f"RAM: {ram_usage}%")

        usage = psutil.disk_usage(self.current_dir)
        free_gb = usage.free / (1024**3)
        self.disk_label.config(text=f"Disco Libre: {free_gb:.2f} GB")

        battery = psutil.sensors_battery()
        if battery:
            self.battery_label.config(text=f"Batería:
{battery.percent}% {'(Cargando)' if battery.power_plugged else ''}")
        else:
            self.battery_label.config(text="Batería: N/A (Sobremesa)")

        self.root.after(2000, self.update_system_stats)
```

```
    def prompt_dialog(self, title, prompt):

        dialog = tk.Toplevel(self.root)
        dialog.title(title)
        dialog.geometry("300x120")
        dialog.transient(self.root)
        dialog.grab_set()

        ttk.Label(dialog, text=prompt).pack(pady=5)
        entry = ttk.Entry(dialog)
        entry.pack(pady=5, padx=10, fill=tk.X)
        entry.focus_set()

        res = {"value": None}
        def ok():
            res["value"] = entry.get()
            dialog.destroy()

        ttk.Button(dialog, text="Aceptar", command=ok).pack(pady=5)
        self.root.wait_window(dialog)
        return res["value"]

if __name__ == "__main__":
    root = tk.Tk()
    app = FileManagerApp(root)
    root.mainloop()
```